

Abstract and Background

Abstract

- With the impending debuts of the Vera C. Rubin Observatory and Nancy Grace Roman Space Telescope, the coming years are expected to produce unprecedented numbers of microlensing events.
 - These telescopes are expected to provide 1.5-10 terabytes of data per day¹, requiring automated processes like machine learning to efficiently parse.
- In this project, we built two distinct machine learning models to predict future flux values for the most common and easily studied type of microlensing event: single-source and single-lens.
- We implemented PyTorch² to test, train, and refine our model utilizing complete light curves from the OGLE³ survey database of past events.

Microlensing

- Microlensing⁴ is a phenomenon where light from a source body bends around a massive object passing between the source and observer.
- Our model intended to fit for: magnification $[A(u)]$, angular separation $[u]$, impact parameter $[u_0]$, time of maximum magnification $[t_0]$, and Einstein radius $[t_E]$.

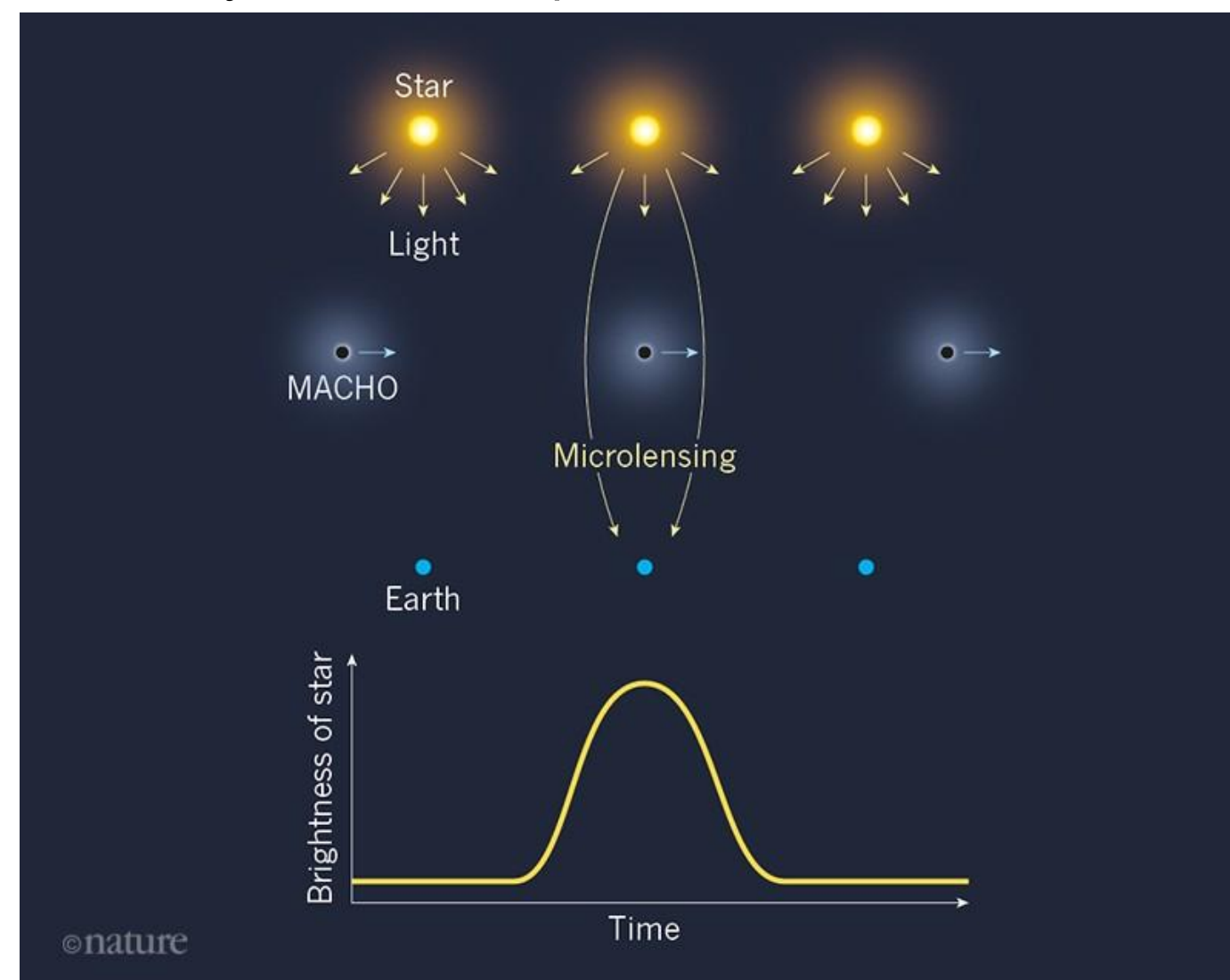


Figure 1: Single-Source Single-Lens Microlensing Event

Machine Learning

- Machine learning is defined as training a computer to complete a task but not explicitly; instead it learns from repeated inputs determining 'correct' outputs.
- PyTorch is a Python package which performs machine learning. It adapts to data and minimizes loss using built-in neural networks⁶. We used this to build a Long-Short Term Memory (LSTM) model.

$$A(u) = \frac{u^2 + 2}{u(u^2 + 4)^{1/2}} \quad u = (u_0^2 + \tau^2)^{1/2} \quad \tau = \frac{(t - t_0)}{t_E}$$

Figure 2: Microlensing Equations

Methodology: Building Our Datasets

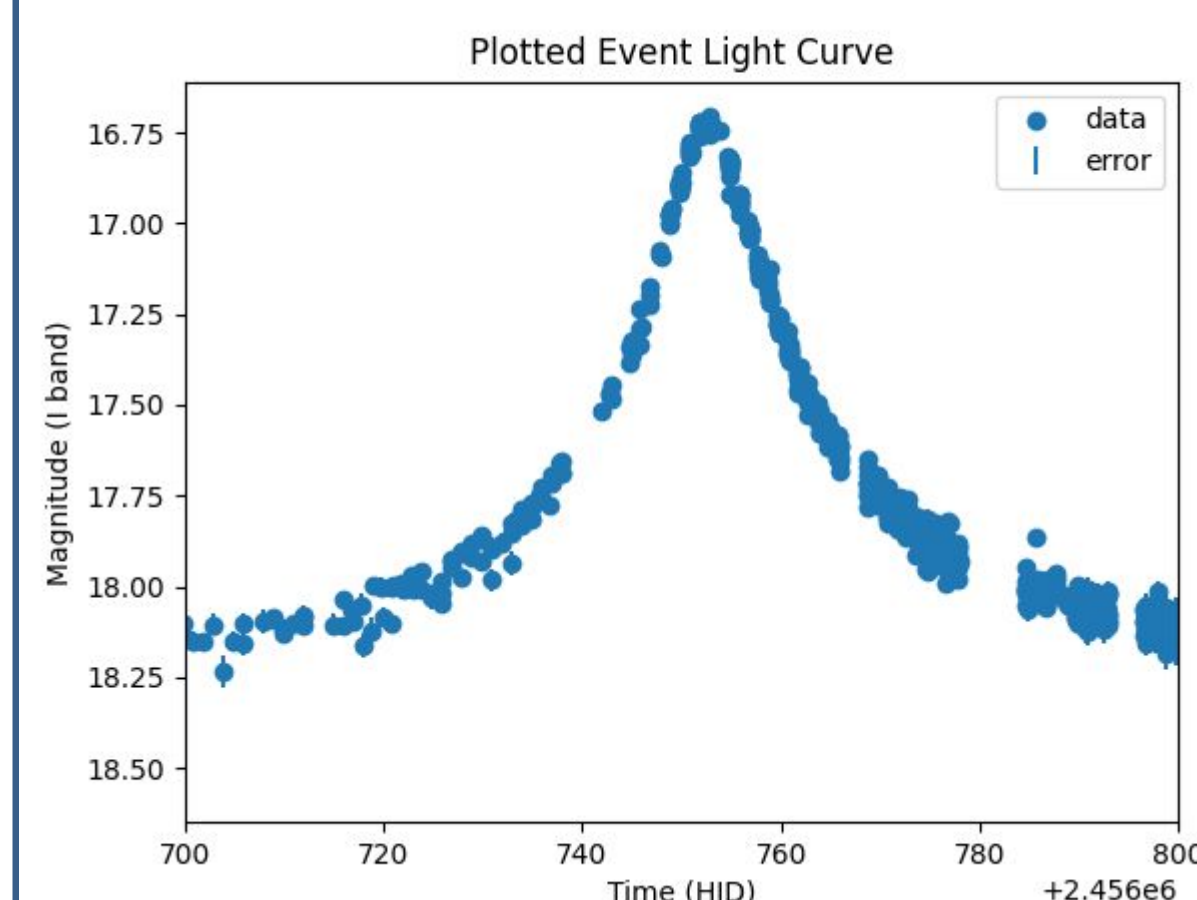


Figure 3: Full Event Light Curve

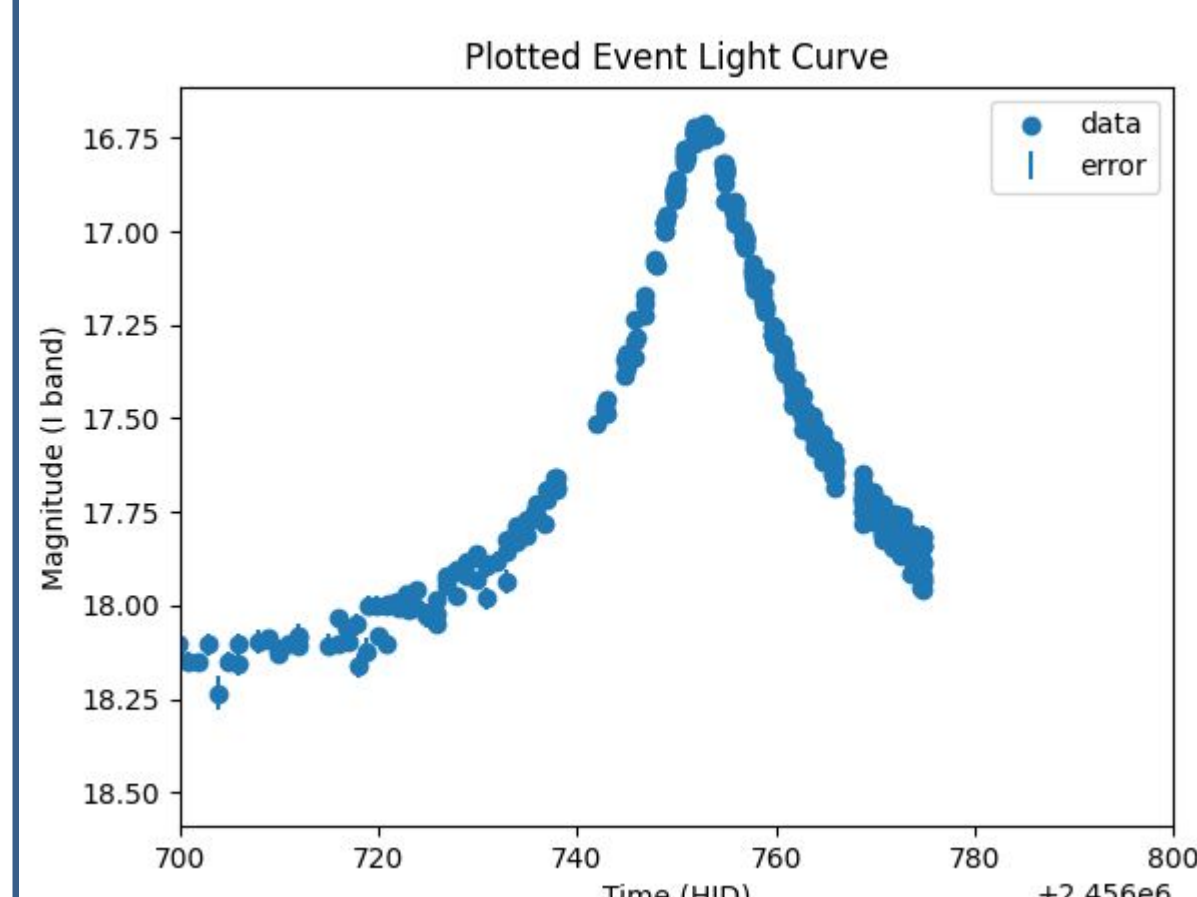


Figure 4: Cutoff Event Light Curve

- Training** is the process where the model learns patterns from labeled data by minimizing prediction error and updating its parameters.
- Testing** is when the model is tested against data it has not seen and its accuracy is measured.
- Initially, we combed through historical OGLE data for well-sampled curves to build a 250-curve training set. From these, we took 20% of the events for our testing set, where we randomly cut off these curves at various points near or after the peak magnitude.
 - The 80/20 split ensured the model was trained on a diverse portfolio of data, while testing enough cutoffs to prove the model could handle various stages of an event.
- In our second iteration we increased our testing/training datasets by developing a filter to automatically parse OGLE for additional well sampled curves.

Methodology: Building the Models

Step #1: Preparing Inputs

- We wrote code to standardize model inputs by both masking gaps in the data and ensuring each data file had consistent tensor sizes.
- After normalizing, the training dataset was also cut off to allow the model to give a predicted output while measuring and minimizing loss with each epoch.

Step #2: Building Our Models

- We chose a mean-squared loss function for both models, but the curve-fitting model had an additional regularization factor.
- Primary model: 3-layer bidirectional LSTM with 256 hidden units, chosen because light curves are sequential data and LSTMs handle variable-length sequences with gaps.
- Additional model: per-event gradient-descent filter that uses Adam to fit the Paczyński parameters (t_0 , t_E , u_0 , fs , and I_0) with no learned prior. Both models output the same Paczyński parameters.

Step #3: Training Models

- The gradient-descent model fits a Paczyński microlensing curve to each event rather than training one model across many events.
- The LSTM was trained for 150 epochs on the 3,810-event training set. Each event was randomly cut off every epoch, and the model with the best test loss was saved.

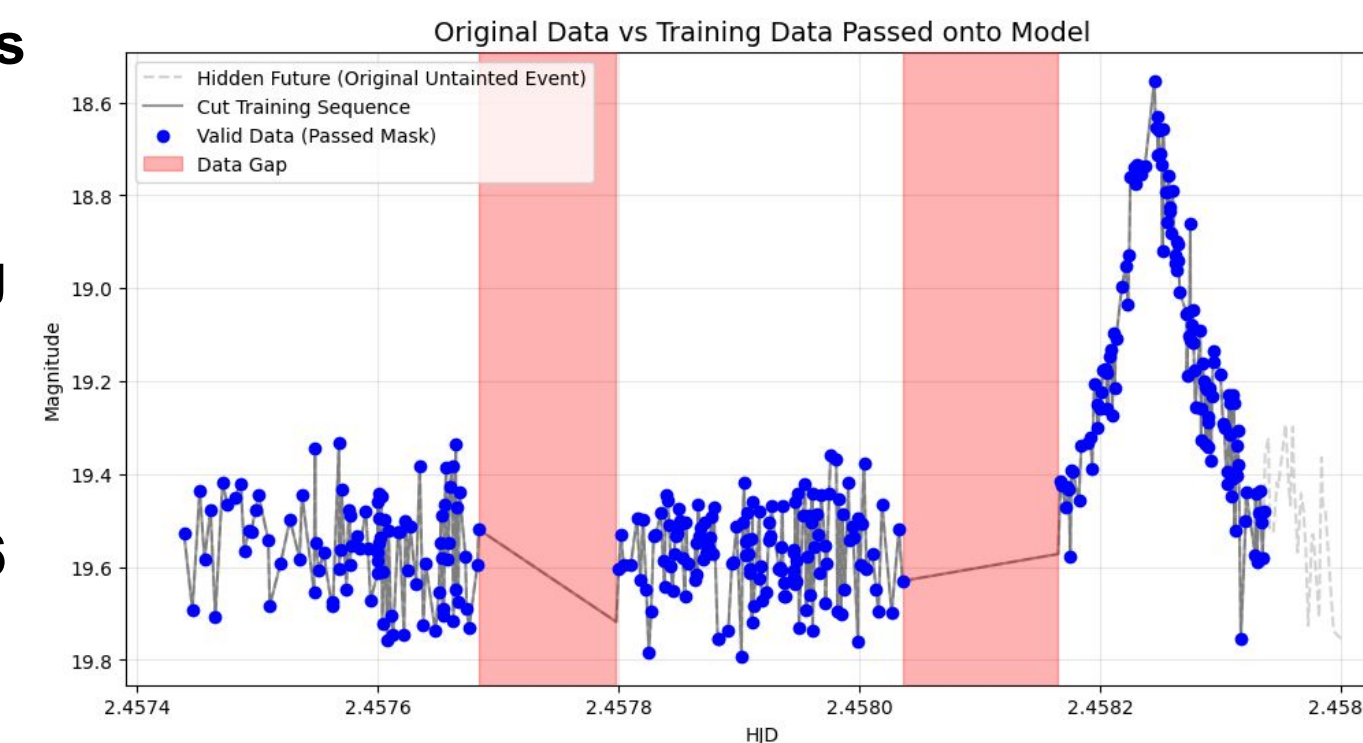


Figure 5: Example of Typical Model Input

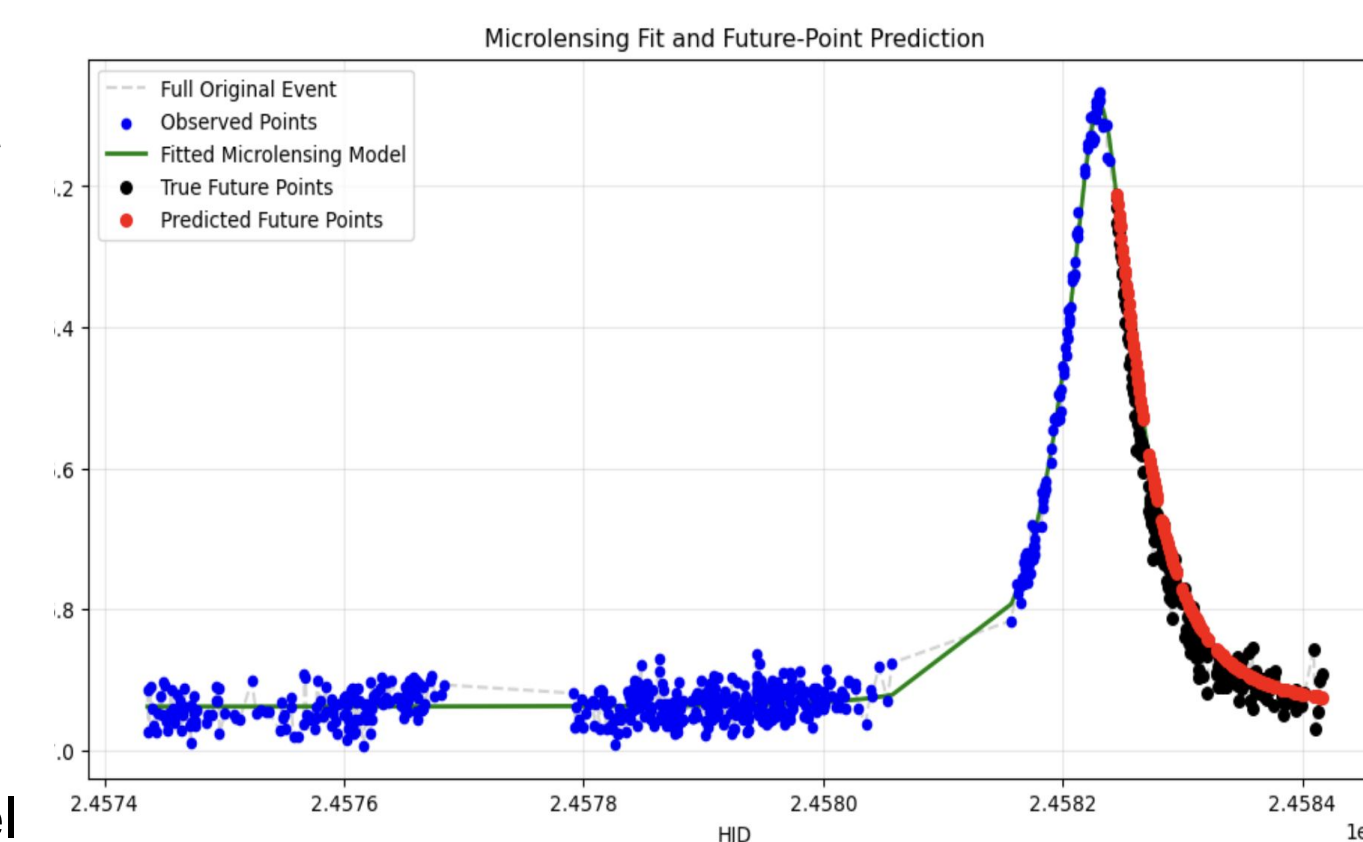


Figure 6: Example of a Gradient Descent Model Output

Step #4: Testing and Refining Models

- The LSTM produced an initial estimate of all five parameters, which is refined with a χ^2 optimizer (scipy L-BFGS-B) which fits the initial estimate to a physics-based Paczyński curve, with a prior penalty to anchor it near the LSTM's guess.
- The gradient descent model performs refinement intrinsically through iterative optimization with a region-weighted loss.

Discussion: Comparing Two Models

- We compared two Paczyński χ^2 optimizers: a gradient-descent baseline with generic individual guesses, and a LSTM-informed optimizer with a learned prior.
- The gradient descent model performs badly if the cutoff happens near or before the peak, since it makes loss uninformative with u_0 , fs , and I_0 , so the optimizer settles near its first guess. This results in clustering near the guesses at $u_0 = 0.2$ and $fs = 0.8$.
- The LSTM + χ^2 optimizer avoids this by using a LSTM prior. It has seen thousands of partial curves, so it produces a guess that is close to the true parameters.
- The χ^2 refinement sharpens this fit without drifting since the χ^2 is guided with a penalty around the LSTM prior. This leads to a mean R^2 of 0.81, with big improvements with u_0 , fs , and I_0 , which were the biggest problems with gradient descent.

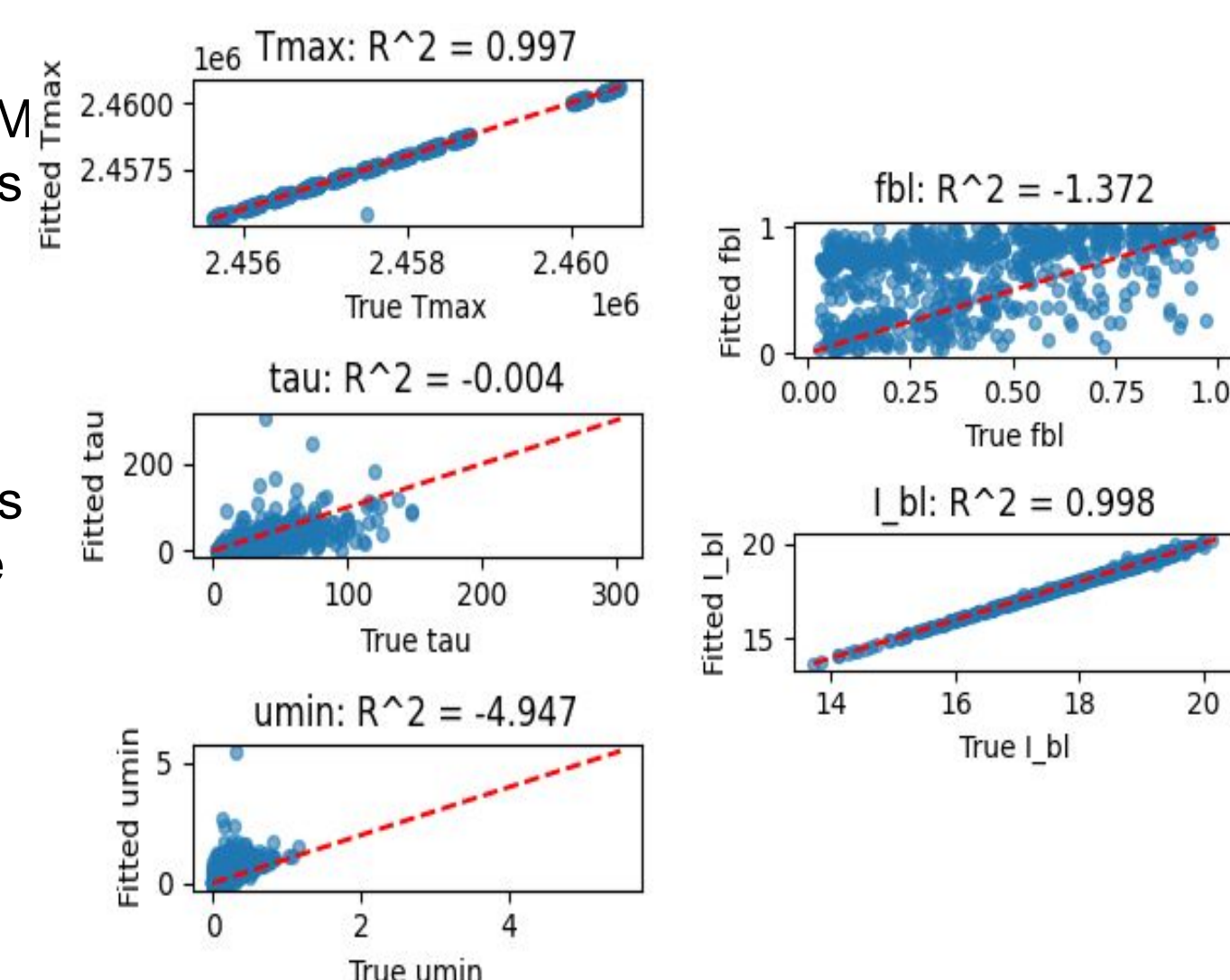


Figure 10: Per-parameter R^2 of gradient descent model. The red line represents the $y=x$ identity.

Results: Predicting Ongoing Events

We evaluated the LSTM + χ^2 model on 673 events, cutting off each curve randomly between the peak and end to simulate a real ongoing event. Our results are shown in Figure 5 and 6, with a total R^2 of 0.809, and a pure-LSTM R^2 of 0.726.

Key Findings:

- We find that t_0 and I_0 were almost perfectly modeled. t_0 was accurate to within one day, and I_0 was accurate to 0.03 magnitudes.
- However, t_E , u_0 , and fs had trouble being recalled. These three parameters are mutually degenerate: a long, faint event with high blending can produce a light curve nearly identical to a short, bright event with low blending.
- The χ^2 optimizer provided a needed refinement using the Paczyński curve equation, unlike the LSTM model. The LSTM's initial guess was essential due to the need of a constraint on the χ^2 filter. Letting the χ^2 filter wander led to a -0.29 u_0 R^2 .
- Failure conditions often happened at extreme parameter values: long t_E values and low u_0 values. These produced narrow peaks and made prediction difficult.

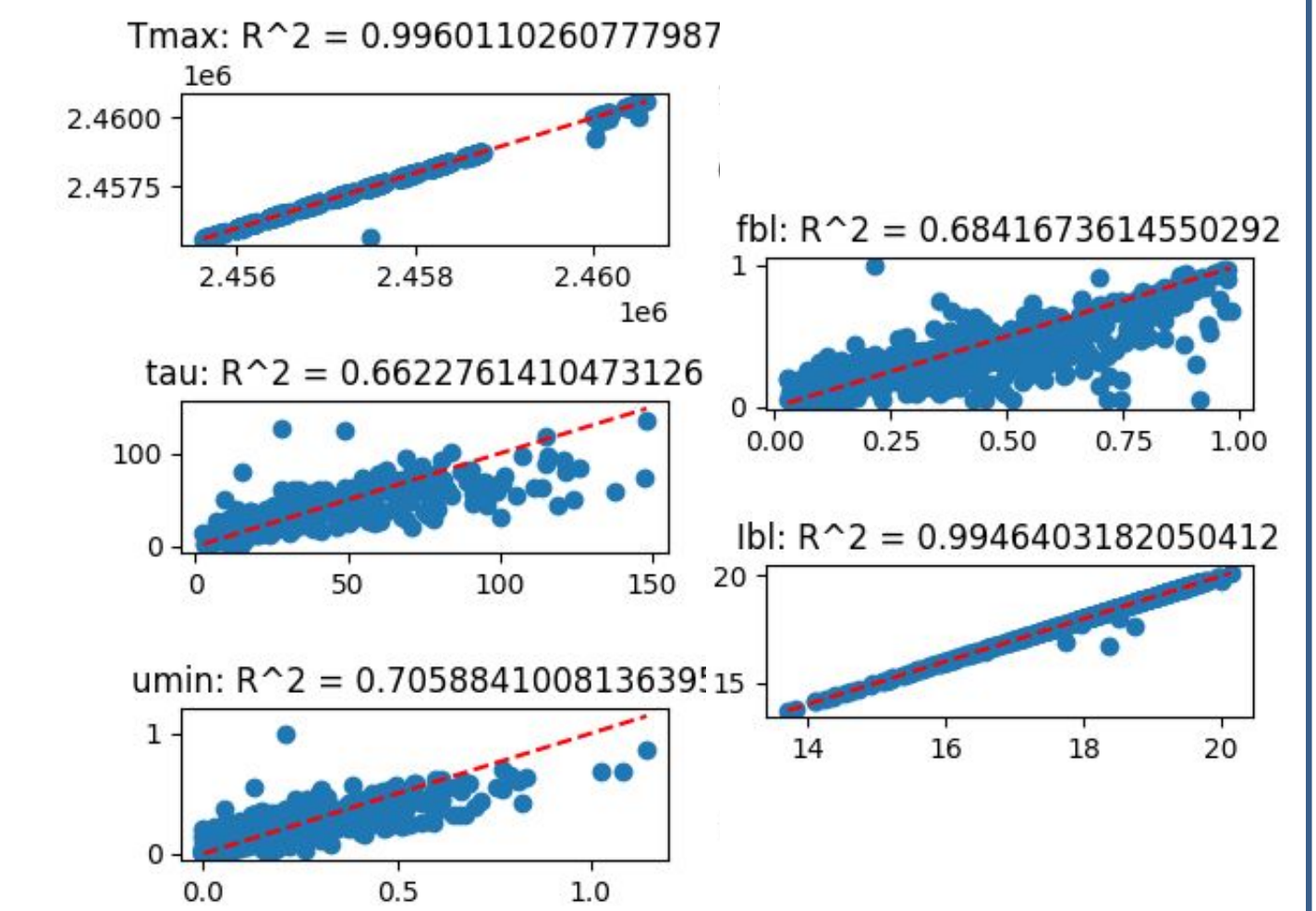


Figure 7: Per-parameter R^2 of LSTM after χ^2 refinement. The red line represents the $y=x$ identity.

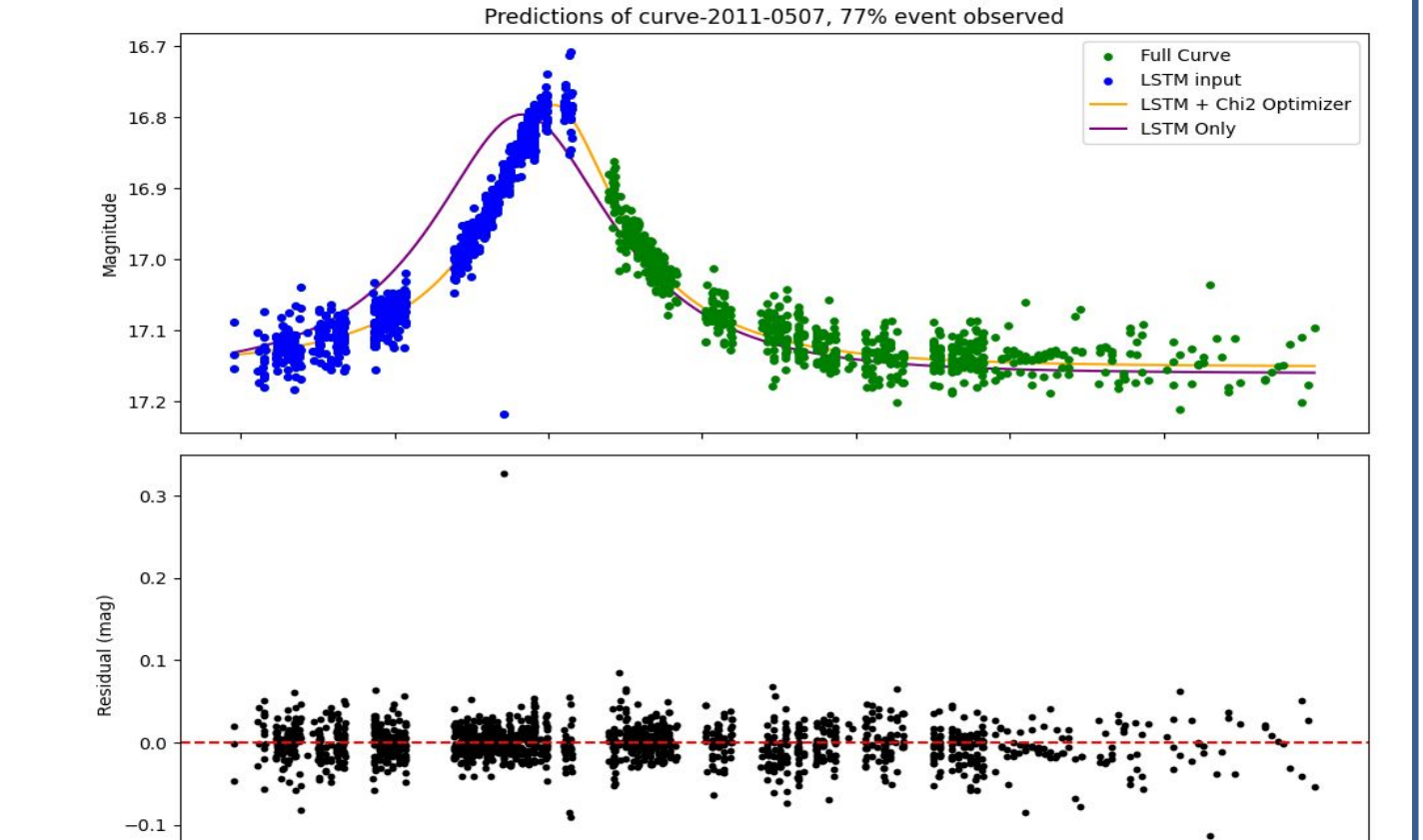


Figure 8: Curve 2011-0507 predicted with LSTM and LSTM with χ^2 model. χ^2 provides a small but needed correction. Lower panel shows the residuals of LSTM + χ^2 model.

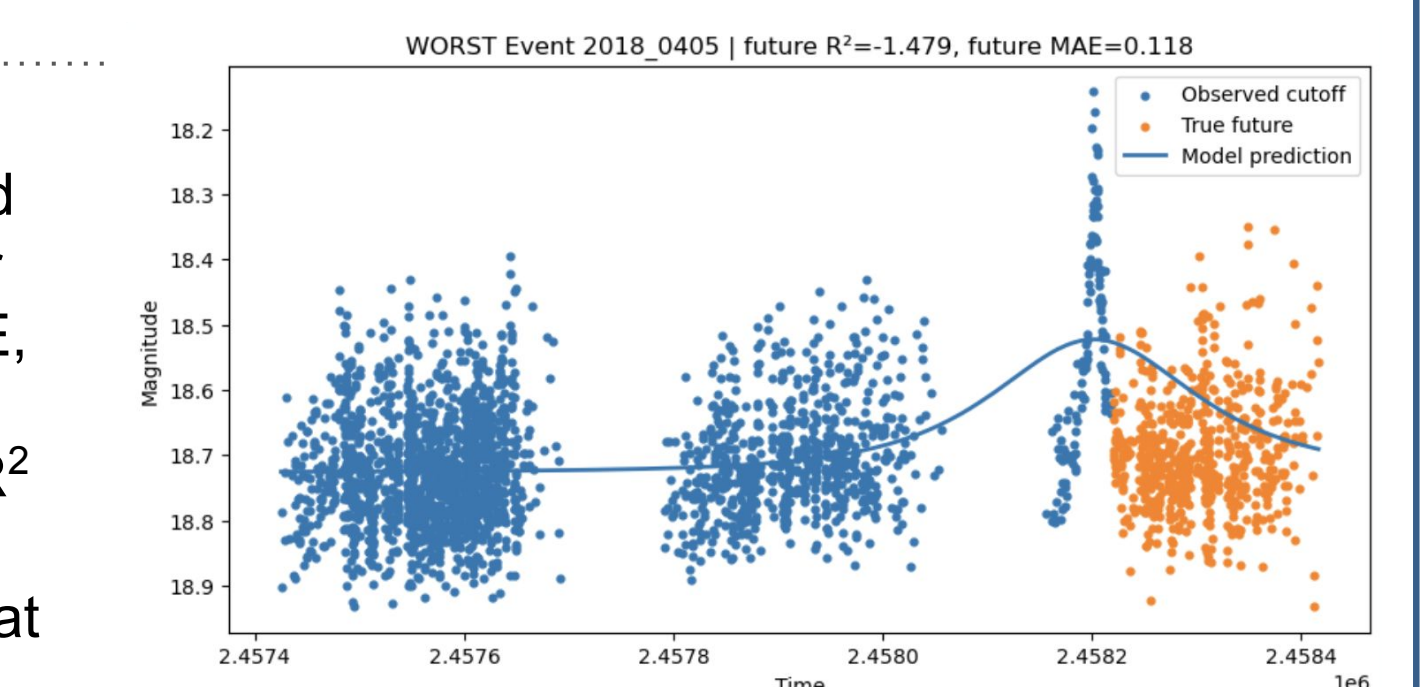


Figure 9: Curve 2018-0405 predicted with the gradient-descent model.

Conclusion and Future Work

Our results show a clear division of labor between the two models. The LSTM recovers t_0 and I_0 accurately (within a day and 0.03 mag respectively) but cannot disentangle the degenerate t_E - u_0 - fs parameters from a partial curve. The χ^2 optimizer fits Paczyński exactly but drifts into unphysical regions without an informed prior ($R^2 = -0.52$). Combining them yields our best result: $R^2 = 0.81$. Going forward, we plan to seed the gradient-descent fitter with the LSTM's initial guess, comparing two strategies for injecting a learned prior into a physics-based fit. With further refinement, models like ours can help handle the incoming data from Rubin/Roman.

References and Acknowledgements

We would like to thank our lovely mentor Caitlin Begbie for her help and guidance throughout these past two semesters. We would also like to thank Undergraduate Labs at Berkeley, Physics and Astronomy, including Directors Andrew McHaty, Yaamini Jois, and Jordan Duan; Lab Managers Olivia Wagner and Sophia Buckner; and Curriculum Managers Brianna Peck and Caitlin Begbie, for giving us the opportunity to gain experience as amateur researchers.

[1] Shude Mao. Introduction to gravitational microlensing, 2008.
 [2] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32, 2019.
 [3] Andrzej Udalski, Michał Krzysztof Szymański, and Grzegorz Szymański. Ogle-iv: fourth phase of the optical gravitational lensing experiment. arXiv preprint arXiv:1504.05966, 2015.
 [4] Rachel Street, Shanan Cross, Jennifer Yee, and Sydney Anderson. Introduction to microlensing — what is microlensing? <https://www.microlensing-source.org/concept/>, 2025. Accessed: 2025-12-03.
 [5] Joachim Wambsganss. Gravitational lensing in astronomy. Living Reviews in Relativity, 1(1):12, 1998.
 [6] Daniel Bourke and freeCodeCamp.org. Pytorch for deep learning & machine learning — full course. <https://www.youtube.com/watch?v=Vxro1bcAuA>, 2020. Accessed : 2025 - 12 - 03.